



Web Application Development

Introduction to the Internet and the WWW

KIEU Quoc Viet HUYNH Vinh Nam

Information and Communication Technology Laboratory (ICTLab),
University of Science and Technology of Hanoi

Hanoi, August 2026



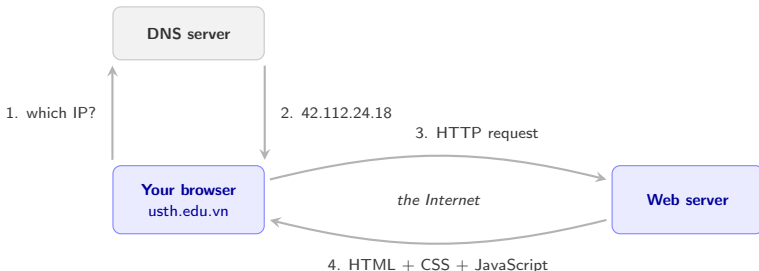
Table of Contents

- 1 Preview
- 2 How we got here
- 3 The Internet underneath
- 4 From Internet to Web

1. Preview

What happens when you open a website?

You do this dozens of times a day without thinking about it.
This whole course lives in the gap between **typing** and **seeing**.



Four arrows, four topics for today.
By the end of this course, **you** write what comes back in step 4.

The Internet is not the Web

Most people use these two words as if they mean the same thing.
They are not. One is the **road**, the other is **one kind of traffic** on it.

The Web

Email

Video calls

Online games

The Internet

cables, routers, IP addresses - since 1969

The Internet

- The physical network and its rules
- Moves bytes between two machines
- Does not care what the bytes mean

The Web

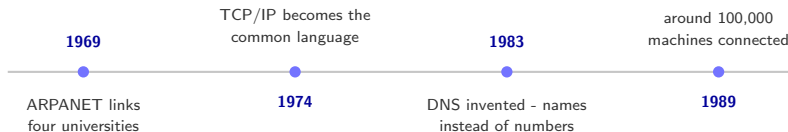
- One application built on that network
- Pages, linked to each other, fetched by a browser
- Arrived in 1990, twenty years later

First half of today: the road. Second half: the traffic **you** are going to build.

2. How we got here

1969–1989: the network before the web

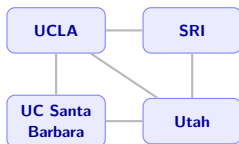
You already know most of these from your networking course.
What matters here is the **order** they arrived in.



By 1989 the network worked well. There was still **nothing to browse**.

1969: four computers and one strange goal

Twenty years before the web, four American universities were wired together.
What they were trying to do explains a lot about how the Internet still behaves.



Built to share machines, not to talk

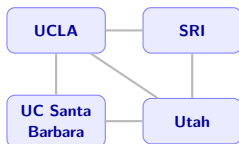
- Computers were rare and expensive - the goal was to run your program on someone else's machine
- Email was never part of the plan. It appeared in 1971 because people wanted it, and became the main traffic within two years

Nobody in charge, on purpose

- Data is cut into packets and each packet finds its own route
- There is no central computer - which is why nobody can switch the Internet off

1969: four computers and one strange goal

Twenty years before the web, four American universities were wired together.
What they were trying to do explains a lot about how the Internet still behaves.



Built to share machines, not to talk

- Computers were rare and expensive - the goal was to run your program on someone else's machine
- Email was never part of the plan. It appeared in 1971 because people wanted it, and became the main traffic within two years

Nobody in charge, on purpose

- Data is cut into packets and each packet finds its own route
- There is no central computer - which is why nobody can switch the Internet off

The first message ever sent was supposed to be **LOGIN**. The system crashed after two letters, so the network's first word was **LO**.

1974: TCP/IP



1974: TCP/IP



IP - the address

- Every machine on the Internet has its own number, the way every house has an address
- IP writes that number on each small piece of data and lets the routers pass it along
- It never checks whether the piece arrived

TCP - the delivery

- Data too big to send at once is cut into small pieces and numbered
- TCP checks which pieces arrived and asks again for the missing ones
- It puts them back in the right order before handing them over

1974: TCP/IP



IP - the address

- Every machine on the Internet has its own number, the way every house has an address
- IP writes that number on each small piece of data and lets the routers pass it along
- It never checks whether the piece arrived

TCP - the delivery

- Data too big to send at once is cut into small pieces and numbered
- TCP checks which pieces arrived and asks again for the missing ones
- It puts them back in the right order before handing them over

Posting a 100-page book through an office that only accepts **one page per envelope**.
IP writes the address on every envelope. TCP numbers the pages, notices page 37 never arrived → asks for it again.
Your requests travel this way - which is why you will never write code to re-send lost data.

1983: DNS



1983: DNS



Domain name - the name

- A short text label people can remember, such as `usth.edu.vn`
- It is only a label. No machine can be reached with it directly

DNS - the lookup service

- Servers that turn a name into the number IP needs
- Your browser asks them **before** sending anything to the site

1983: DNS



Domain name - the name

- A short text label people can remember, such as `usth.edu.vn`
- It is only a label. No machine can be reached with it directly

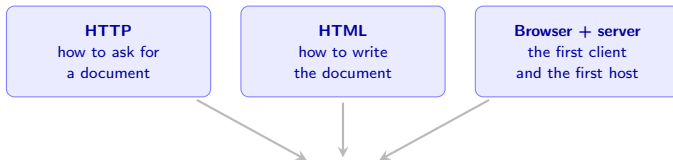
DNS - the lookup service

- Servers that turn a name into the number IP needs
- Your browser asks them **before** sending anything to the site

The contact list on your phone. You type **Mum**, the phone finds the number, the call goes to the number.
You could always dial the number yourself - and you can type 42.112.24.18 instead of `usth.edu.vn`.
By 1989 naming, addressing and delivery all worked. There was still **nothing to browse**.

1990: one person, three inventions

Tim Berners-Lee, a physicist at CERN, wanted to link research documents together.
In one year he built **all three pieces** the web still runs on.

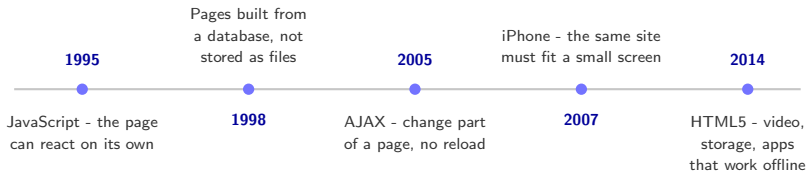


All three are still in use today - you will write code for each of them

In 1993 the Mosaic browser added images inside the page.
That is the moment the web stopped being for researchers.

1995–today: from pages to applications

The web started as documents you **read**.
Every step below pushed it closer to software you **use**.



Most of what you open in a browser today is not a document at all. It is an **application**.
Building one is what the rest of this course is about.

3. The Internet underneath

What a protocol is

You already know the word from your networking course.
What matters here is **how many of them** you are going to touch this semester.

The rules of a conversation

- What a message looks like, and in what order messages come
- Both sides must follow the same rules, or neither one understands the other
- Buying coffee: you say the drink, they say the price, you pay, they hand it over. Swap two steps and it falls apart

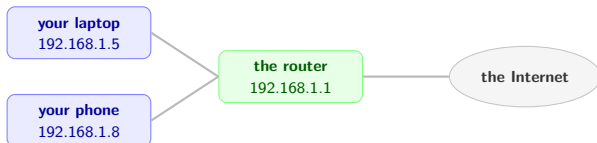
The ones you will meet

- **HTTP** - ask a server for a page
- **HTTPS** - the same, but nobody in between can read it
- **DNS** - turn a name into a number
- **TCP** - deliver the pieces in order
- **SMTP** - send email

Most of this course is **one** protocol: HTTP.
Everything else today is there so that HTTP makes sense.

What an IP address is

A network finds a device by a **number**, never by a name.
There are already a few hundred of these numbers in this room.



Reading the number

- Four numbers, each from 0 to 255, separated by dots
- Handed out by the router when a device joins.
Connect at home tonight and you get a different one

Two worth recognising

- 127.0.0.1 - always this machine, on every computer ever made
- 192.168.1.1 - open it in a browser, your router asks for a password

Run `ipconfig` on Windows, or `ip addr` on Mac and Linux, and read your own number.
Keep it - we come back to it shortly, because it is not the number the Internet sees.

IPv4 vs IPv6

Four numbers of 0–255 give about **4.3 billion** addresses.

That looked endless in 1981. Our region ran out of free addresses in **April 2011**.

	IPv4	IPv6
Looks like	42.112.24.18	2001:db8:85a3::8a2e:370:7334
Written with	four numbers, dots	eight groups, colons, hexadecimal
Size	32 bits	128 bits
How many	about 4.3 billion	about 3.4×10^{38}
In use since	1981	1998, slowly

IPv4

- Four numbers and three dots. Every network on Earth already understands it
- **Good:** short enough to read out loud and type from memory
- **Cost:** short also means few - only 4.3 billion exist, and they ran out

IPv6

- Eight groups of letters and digits, separated by colons
- **Good:** that length gives enough addresses for every device, forever
- **Cost:** that same length is unreadable, so 25 years later many networks still refuse it

IPv4 did not disappear - it survives by letting many machines share one address.

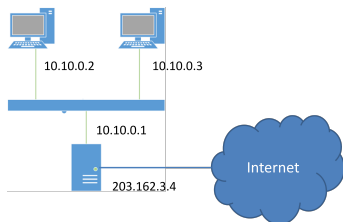
That trick is the next slide.

Virtual vs real IP address

Virtual - used inside one network only. Starts with 192.168, 10. or 172.16–31

Real - unique on the whole Internet. One per household or office

Search **what is my IP** - the number you get back is not the one ipconfig



Follow one request

1. Your computer is 10.10.0.2. It asks for a web page.
2. The router crosses out 10.10.0.2, writes its own 203.162.3.4 instead, and keeps a note: *10.10.0.2 asked for this*.
3. The web site answers to 203.162.3.4. It never learns that your computer exists.
4. The router reads its note and passes the answer to 10.10.0.2.

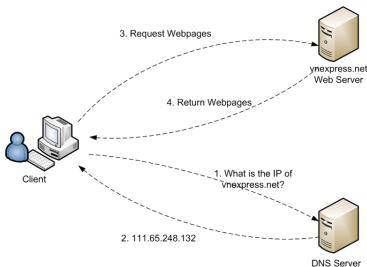
What this buys you, and what it costs

- **Good:** one office needs one real address instead of a hundred. This is why IPv4 still works today
- **Cost:** sharing one address means sharing the blame. If one person here misbehaves, a site can block the whole building
- **Good:** nobody outside can reach your computer. It is hidden without you doing anything
- **Cost:** that wall works both ways - you cannot host anything either

Domain names and DNS

A browser cannot use a name. It needs a **number**.

So before asking for the page, it has to ask someone else for the number.



You type vnexpress.net

1. **Browser to DNS server:** *what is the number for vnexpress.net?*
2. **DNS server back:** *111.65.248.132*
3. **Browser to 111.65.248.132:** *send me the page*
4. **Web server back:** *here it is*

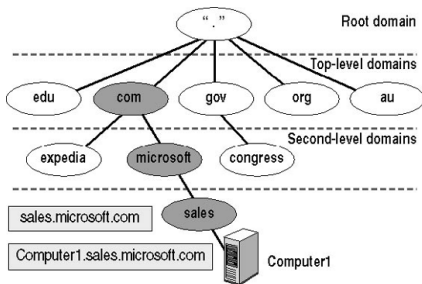
Two machines, two jobs.

The DNS server has not seen the page, it only gives directions.
A slow DNS server makes every site feel slow, even a perfect one.

The DNS tree

No single machine could hold every name on the Internet.

So names are split into levels, and each level only knows about the one below it.



Read a name from right to left

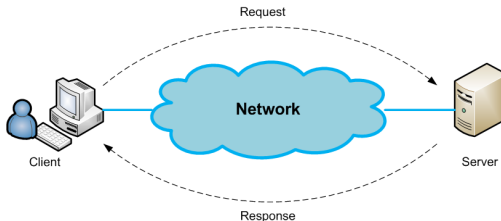
- vn - the country. Managed by Vietnam
- edu.vn - schools in Vietnam. Managed under .vn
- usth.edu.vn - the part the university registered
- mail.usth.edu.vn - USTH decides this part alone, no permission needed

Why it is built this way

- Each level hands the question down instead of answering everything itself
- Buying a domain means renting one name under an existing level - and then every level below it is yours

Client and server

These are not two kinds of computer. They are two **roles** in one conversation.
The role is decided by who speaks first.



Client - asks

- Starts every conversation. Waits for an answer, then usually stops
- Your browser, your phone app, a program calling an API

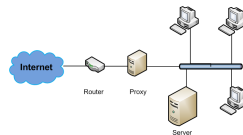
Server - answers

- Never speaks first. Waits, and answers thousands of clients at the same time
- Must always be reachable, so it needs a real address

In the lab your laptop plays **both** roles: a server program answering on 127.0.0.1, and a browser asking it.
That is why you can build a whole web site before putting anything online.

Firewall and proxy

Both sit between you and the Internet, and both look the same in a diagram.
But one **stops** messages, and the other **carries** them for you.



Firewall - the guard at the door

- Lets through only what is allowed. Not on the list, no entry
- **Good:** an attack is stopped before it reaches anything
- **Cost:** it blocks your own work too, until someone opens that door on purpose

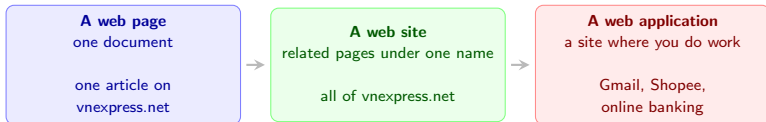
Proxy - the friend you send

- You tell the proxy, it asks the server, it brings the answer back. The shop sees your friend, not you
- **Good:** hides who asked, and keeps a copy of popular pages so they arrive faster
- **Cost:** sometimes slow due to multiple requests; might lost data or being attacked

A firewall answers *is this allowed?* - a proxy answers *who is doing the asking?*

4. From Internet to Web

Page, site, application



On a page you read

- You open it, you scroll, you leave
- Nothing you did is kept. Come back tomorrow and the page has no idea you were here

In an application you use

- You log in, you type, you click send, you buy something
- What you did is saved on a server. Open Gmail tomorrow and your messages are still there

HTTP and HTML

HTML - HyperText Markup Language

```
<h1>Course list</h1>
<p>Web Application Development</p>
```

A language for writing the document

- Tags are put around text to say what each part is: a heading, a paragraph, a link
- **You write this.** It is the page

HTTP - HyperText Transfer Protocol

```
GET /courses HTTP/1.1
Host:  usth.edu.vn
```

A set of rules for moving it

- The rules governing the conversation between a Web client and a Web server
- **You do not write this.** The browser and the server follow it

HTTP response

the HTML goes in here

Same person,
same year: 1990



One HTTP conversation

The browser asks

```
GET /courses/web HTTP/1.1
Host: usth.edu.vn
```

Reading the request

- GET - fetch me something
- /courses/web - which page on that server
- Host - which site, because one server holds many sites

The server answers

```
HTTP/1.1 200 OK
Content-Type: text/html

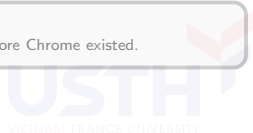
<h1>Course list</h1>
```

Reading the answer

- 200 - found it. 404 - no such page
- Content-Type - what is coming: a page, a picture, a video
- After the blank line: the HTML itself

These two shapes were fixed in 1990 and never changed.

Chrome, released in 2008, can fetch a page from a server program written before Chrome existed.



Two words in every conversation

The method — what you want done

GET	read	opening any page
POST	create	pressing Sign up
PUT	replace	saving an edited profile
DELETE	remove	deleting a photo

You have met these four before

- In your database course they were called
SELECT, INSERT, UPDATE, DELETE

The status — how it went

2xx	200 OK	here is what you asked for
	201 Created	your Sign up worked
3xx	301 Moved	the address changed for good
	304 Not Modified	use the copy you already have
	400 Bad Request	the server cannot read it
4xx	401 Unauthorized	log in first
	403 Forbidden	logged in, still not allowed
	404 Not Found	no such page here
5xx	500 Server Error	the program crashed
	503 Unavailable	too busy, or being updated

The one you will see most this term

- 500 — and the program that crashed will be yours

4xx → look at **my** request · 5xx → look at **the** server

Web client: the browser

In every conversation we have drawn today, the **client** is your web browser.
Chrome, Safari, Edge, Firefox, Opera, Coc Coc - all of them send the same GET line.



What the program does

- Sends the GET line to the server
- Reads the HTML that comes back
- Draws it on the screen, and asks again for every picture the HTML names

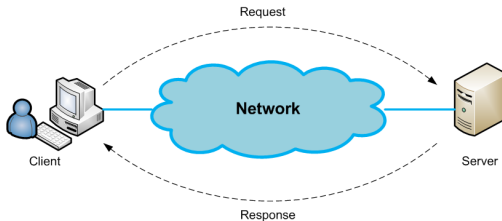
Watch it happen yourself

- Press **F12**, open the **Network** tab, reload the page
- Every request from the last slide is listed there: the path, the 200 or 404, and what came back

Web server

A web server is a **program**, not a machine.

Apache, Nginx, Internet Information Services, Node.js - you install one, the way you install any program.



What the program does

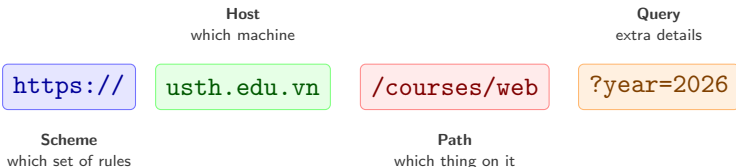
- Waits. It never speaks first
- Reads the path in the request and looks for what was asked for
- Sends it back with 200, or sends 404 if it is not there

You will run one this week

- Install Node.js on your laptop and your laptop is a web server
- Open 127.0.0.1 in Chrome and your laptop is also the client

Anatomy of a URL

A URL is not one thing. It is four answers to four different questions, glued together and typed into one box.



Decided for you

- 1 `https` is a protocol everyone agreed on long ago
- 2 `usth.edu.vn` was registered once, and DNS turns it into a number

Decided by you

- 3 The **path** is invented by whoever built the site. There is no rule that `/courses` must exist
- 4 The **query** carries values into the request - a search word, a page number, a year

Home page, hyperlink, tab



Each arrow is
one hyperlink

A hyperlink is one HTML tag

```
<a href="/courses">See all courses</a>
```

What it does

- The reader clicks the words *See all courses*
- The browser sends GET /courses and draws whatever comes back

Home page

- Type `usth.edu.vn` with no path, and the server sends the home page
- It is the page every other page is reached from

Tabs

- Ten tabs means ten separate conversations, with ten different servers, at the same time

Web multimedia

A page is not only text. The same GET request can bring back a song, a film or a map.
The Content-Type line tells the browser which one is arriving.

Four kinds, and where you meet them

- **Audio** - Spotify playing in a tab
- **Video** - YouTube
- **Animation** - the spinning circle while a page loads
- **3D** - the buildings in Google Maps

Playing a video takes one tag

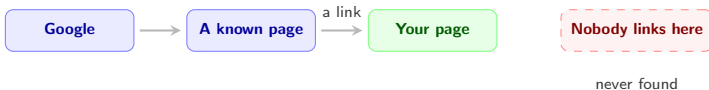
```
<video src="lecture.mp4" controls>  
</video>
```

Why this matters

- Before HTML5, a video needed a separate program installed in the browser
- Now the browser plays it. Nothing to install, on any machine

Search engines

Google, Bing, Coc Coc - programs that find pages on the WWW for you.
They find them the same way you do: by following links.



How a search engine finds a page

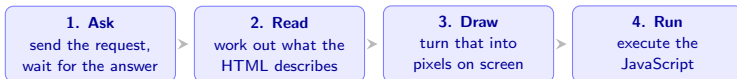
- A program of Google's sends GET to a page it already knows
- It reads every `<a href>` in the HTML and sends GET to those too
- It repeats this, day and night, across the whole web

What this means for your project

- A page that no other page links to is a page Google never reaches
- The hyperlink from the last slide is not only how readers move - it is how your site gets found at all

What a browser actually does

A browser is not a window. It is a program that does **four jobs** every time you open a page.
Three of them you never see.



One page is not one request

- The HTML arrives first. It only **mentions** the images, the styles, the scripts
- The browser then asks for each one separately. A normal page costs 50 to 100 requests

Step 4 is unusual

- Once drawn, the JavaScript keeps running and can ask for more, with no reload
- That is the whole difference between a page you read and an application you use

Thank you for listening!



Scan for the session evaluation!